

BÀI GIẢNG: NGẮT TRONG STM32F103C8

1. Khái niệm và vai trò của ngắt (Interrupt)

- Ngắt là tín hiệu từ phần cứng hoặc phần mềm yêu cầu CPU tạm ngưng chương trình đang chạy để xử lý sự kiện khẩn cấp hoặc quan trọng.
- Khi có ngắt, CPU sẽ:
 - Tạm dừng chương trình chính (main loop).
 - Nhảy vào hàm xử lý ngắt (ISR - Interrupt Service Routine).
 - Sau khi xử lý, quay lại tiếp tục chương trình chính.
- Ngắt giúp chương trình:
 - Phản ứng kịp thời với sự kiện bên ngoài.
 - Giảm lãng phí CPU do chờ đợi (polling).

2. Cấu trúc ngắt trên STM32F103C8

2.1 Vi xử lý Cortex-M3 và NVIC

- STM32F103C8 dùng vi xử lý ARM Cortex-M3.
- Có bộ điều khiển ngắt tích hợp là **NVIC** (Nested Vectored Interrupt Controller).
- NVIC quản lý nhiều nguồn ngắt (ngoại vi, hệ thống).
- Có 2 nhóm ngắt chính:
 - **Exception**: Ngắt hệ thống như Reset, NMI, HardFault, SysTick,...
 - **Interrupt**: Ngắt từ các ngoại vi như Timer, EXTI, UART,...

2.2 Vector ngắt

- Mỗi nguồn ngắt có một vector ngắt (địa chỉ hàm ISR) trong bảng Vector Table.
- Ví dụ:
 - SysTick_Handler cho ngắt SysTick.
 - TIM1_UP_IRQHandler cho ngắt Timer 1 update.
 - EXTI15_10_IRQHandler cho ngắt các chân EXTI 10 đến 15.

2.3 Ưu tiên ngắt

- NVIC cho phép gán ưu tiên ngắt (Priority), giúp xử lý ngắt quan trọng trước.
- Có thể **bật tắt ngắt riêng biệt** từng nguồn.

3. Cách cấu hình ngắt chung trên STM32F103C8

3.1 Bước 1: Cấu hình ngoại vi gây ngắt

- Ví dụ Timer, GPIO, UART,...

- Kích hoạt nguồn clock cho ngoại vi đó.

3.2 Bước 2: Cấu hình nguồn ngắt trên NVIC

- Dùng hàm hoặc thao tác thanh ghi để:
 - Bật enable ngắt.
 - Đặt mức ưu tiên (nếu cần).

3.3 Bước 3: Viết hàm xử lý ngắt (ISR)

- Tên hàm đúng chuẩn startup file (vector table).
- Trong hàm cần xóa cờ ngắt (flag) để không bị lặp lại.

3.4 Bước 4: Bật ngắt toàn cục (global interrupt)

- Thường dùng hàm `__enable_irq()` hoặc bật trong thanh ghi PRIMASK.

4. Ngắt SysTick

4.1. Giới thiệu

- **SysTick** là một bộ đếm 24-bit được tích hợp sẵn trong lõi **Cortex-M3** của STM32F1.
- Đặc điểm:
 - Đếm xuống từ giá trị tải (LOAD) đến 0.
 - Khi đếm đến 0, tự động nạp lại giá trị LOAD và tạo **sự kiện ngắt** nếu được cho phép.
 - Tích hợp sẵn, không cần bật clock từ RCC.
- **Ứng dụng thường gặp:**
 - Tạo thời gian định kỳ (1ms, 1s, ...)
 - Sinh "tick" cho hệ điều hành thời gian thực (RTOS)
 - Đo thời gian hoặc delay chính xác.

4.2. Cấu hình SysTick tạo ngắt

4.2.1. Chọn nguồn clock

Thanh ghi **SYST_CSR** (Control and Status Register)

- Bit 2 **CLKSOURCE**:
 - 0: Clock từ **AHB/8**
 - 1: Clock từ **AHB** (HCLK)
- Trong STM32F1, thường chọn **AHB** để có độ phân giải cao.

`SysTick->CTRL |= SysTick_CTRL_CLKSOURCE_Msk; // Chọn HCLK = 72 MHz`

4.2.2. Nạp giá trị đếm

Thanh ghi **SYST_RVR** (Reload Value Register)

- Giá trị reload: từ **1** đến **0xFFFFFFFF** (24-bit).
- Tần số ngắt:

$$f_{ngat} = f_{clock} \cdot \frac{Reload + 1}{2^{24}}$$

Ví dụ 1ms:

$$SysTick \rightarrow LOAD = 72000 - 1; // 72 \text{ MHz} / 72000 = 1 \text{ kHz} \rightarrow \text{ngắt mỗi 1ms}$$

4.2.3. Reset bộ đếm về 0

Thanh ghi **SYST_CVR** (Current Value Register)

- Ghi bất kỳ giá trị nào $\rightarrow$ reset về 0.

`SysTick->VAL = 0;`

4.2.4. Bật ngắt SysTick

Thanh ghi **SYST_CSR**

- Bit 1 **TICKINT**:
 - 0: Không tạo ngắt
 - 1: Cho phép ngắt khi đếm hết

`SysTick->CTRL |= SysTick_CTRL_TICKINT_Msk;`

4.2.5. Bật bộ đếm SysTick

Thanh ghi **SYST_CSR**

- Bit 0 **ENABLE**:
 - 0: Dừng đếm
 - 1: Bắt đầu đếm

`SysTick->CTRL |= SysTick_CTRL_ENABLE_Msk;`

4.3. Chương trình xử lý ngắt (ISR)

```
volatile uint32_t systick_count = 0;
```

```
void SysTick_Handler(void) {
```

```
    systick_count++; // Mỗi 1ms tăng 1
```

```
}
```

- ISR này được gọi mỗi lần bộ đếm SysTick tràn.
- `SysTick_Handler` là tên hàm chuẩn do CMSIS định nghĩa.

4.4. Thuật toán hoạt động

1. Bắt đầu
2. Chọn nguồn clock (HCLK)
3. Gán giá trị LOAD (ví dụ 72000 - 1 cho 1ms)
4. Reset VAL = 0
5. Bật ngắt (TICKINT = 1)
6. Bật bộ đếm (ENABLE = 1)
7. Chờ sự kiện đếm hết
8. Khi đếm hết:
 - CPU nhảy vào SysTick_Handler
 - Tăng biến đếm hoặc xử lý tác vụ định kỳ
9. Quay lại bước 7.

4.5. Ví dụ đầy đủ

```
#include "stm32f10x.h"

volatile uint32_t ms_ticks = 0;

void SysTick_Init(void) {
    // Cấu hình SysTick 1ms
    SysTick->LOAD = 72000 - 1;           // 72 MHz / 72000 = 1 kHz
    SysTick->VAL = 0;                    // Reset bộ đếm
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk // Chọn HCLK
        | SysTick_CTRL_TICKINT_Msk // Bật ngắt
        | SysTick_CTRL_ENABLE_Msk; // Bật bộ đếm
}

void SysTick_Handler(void) {
    ms_ticks++;
    if (ms_ticks % 1000 == 0) { // Mỗi 1 giây
        GPIOC->ODR ^= (1 << 13);
    }
}

int main(void) {
    // Bật clock GPIOC
    RCC->APB2ENR |= (1 << 4);
```

```

// Cấu hình PC13 output
GPIOC->CRH &= ~(0xF << 20);
GPIOC->CRH |= (0x2 << 20);
SysTick_Init();
while (1) {
    // Vòng lặp chính rảnh
}
}

```

4.6. Ứng dụng

- Đo thời gian chính xác ở cấp độ ms
- Sinh tick cho RTOS (FreeRTOS, CMSIS-RTOS)
- Tạo delay không chặn (delay_ms)
- Thực hiện tác vụ định kỳ mà không ảnh hưởng đến chương trình chính.

5. Ngắt Timer 1 (TIM1)

5.1. Giới thiệu

- **TIM1** là **advanced-control timer** trên STM32F1, có thể hoạt động ở nhiều chế độ: tạo PWM, đo tần số, tạo thời gian định kỳ, hoặc tạo ngắt theo chu kỳ.
- Sự kiện ngắt: ngắt Timer 1 xảy ra khi bộ đếm Tim1_CNT tăng đến giá trị thanh ghi **ARR** và quay lại 0.
- Ứng dụng thường gặp:
 - Đếm thời gian định kỳ (1ms, 1s, ...)
 - Gọi hàm xử lý theo chu kỳ cố định
 - Điều khiển định thời các tác vụ

5.2. Cấu hình Timer 1 tạo ngắt

5.2.1 Bật clock cho TIM1

RCC->APB2ENR |= RCC_APB2ENR_TIM1EN;

Hoặc RCC->APB2ENR |= (1 << 11);

Thanh ghi liên quan: RCC->APB2ENR (APB2 Peripheral Clock Enable Register)

- Dùng để bật/tắt clock cho các ngoại vi nằm trên bus **APB2**.
- Bit điều khiển TIM1:
 - **Bit 11 – TIM1EN:**
 - 0: Tắt clock TIM1 (Timer1 không hoạt động)
 - 1: Bật clock TIM1 (Timer1 bắt đầu hoạt động)

5.2.2 Cấu hình tần số Timer

Các thanh ghi chính

1. PSC (Prescaler Register)

- Chia tần số clock timer trước khi đưa vào bộ đếm.
- Giá trị chia = PSC + 1.
- Ví dụ: Clock APB2 = 72 MHz, PSC = 7199 → tần số đếm của Timer1 = $72 \text{ MHz} / (7199 + 1) = 10 \text{ kHz}$.

2. ARR (Auto-Reload Register)

- Giá trị tối đa bộ đếm trước khi reset về 0.

- Nếu $ARR = 9999 \rightarrow$ Timer 1 đếm từ 0 đến 9999 rồi bị tràn, reset về 0, tạo ra sự kiện ngắt.

Ví dụ cấu hình 1s:

$TIM1 \rightarrow PSC = 7200 - 1;$ // Chia 72 MHz $\rightarrow$ 10 kHz

$TIM1 \rightarrow ARR = 10000 - 1;$ // $10\text{ kHz} * 1\text{ s} = 10,000$ xung

Kết quả: Cứ mỗi 1 giây, Timer 1 tạo ra 1 sự kiện ngắt.

5.2.3 Bật ngắt Update

$TIM1 \rightarrow DIER |= TIM_DIER_UIE;$

Hoặc $TIM1 \rightarrow DIER |= (1 << 0);$ // Set bit 0 của DIER

Thanh ghi DIER (DMA/Interrupt Enable Register)

- **UIE** (Update Interrupt Enable, bit 0):
 - 0: Không cho phép ngắt Timer 1
 - 1: Cho phép ngắt Timer 1 khi sự kiện ngắt xảy ra

5.2.4 Bật Timer 1

$TIM1 \rightarrow CR1 |= TIM_CR1_CEN;$

Hoặc $TIM1 \rightarrow CR1 |= (1 << 0);$ // Set bit 0 của CR1

Thanh ghi CR1 (Control Register 1)

- **CEN** (Counter Enable, bit 0):
 - 0: Dừng bộ đếm
 - 1: Bắt đầu đếm

5.2.5 Bật ngắt trong NVIC

`NVIC_EnableIRQ(TIM1_UP_IRQn);`

- **TIM1_UP_IRQn** là tên định nghĩa trong file header của STM32 cho ngắt update của Timer 1.
- NVIC quản lý các ngắt của CPU, cần bật ở đây để CPU nhận ngắt từ TIM1.

5.3. Chương trình xử lý ngắt (ISR)

```

void TIM1_UP_IRQHandler(void) {
    if (TIM1->SR & TIM_SR_UIF) { // Kiểm tra cờ update
        TIM1->SR &= ~TIM_SR_UIF; // Xóa cờ ngắt (UIF = 0)
        timer1_tick++; // Xử lý: tăng biến đếm
    }
}

/*
    if (TIM1->SR & (1 << 0)) { // Kiểm tra bit 0 (UIF)
        TIM1->SR &= ~(1 << 0); // Xóa bit 0
        timer1_tick++;
    }
}
*/

```

Theo như cài đặt ở trên, sau đúng 1s, sự kiện ngắt xảy ra,
 Vào đây tăng biến đếm timer1_tick để định bao nhiêu s.

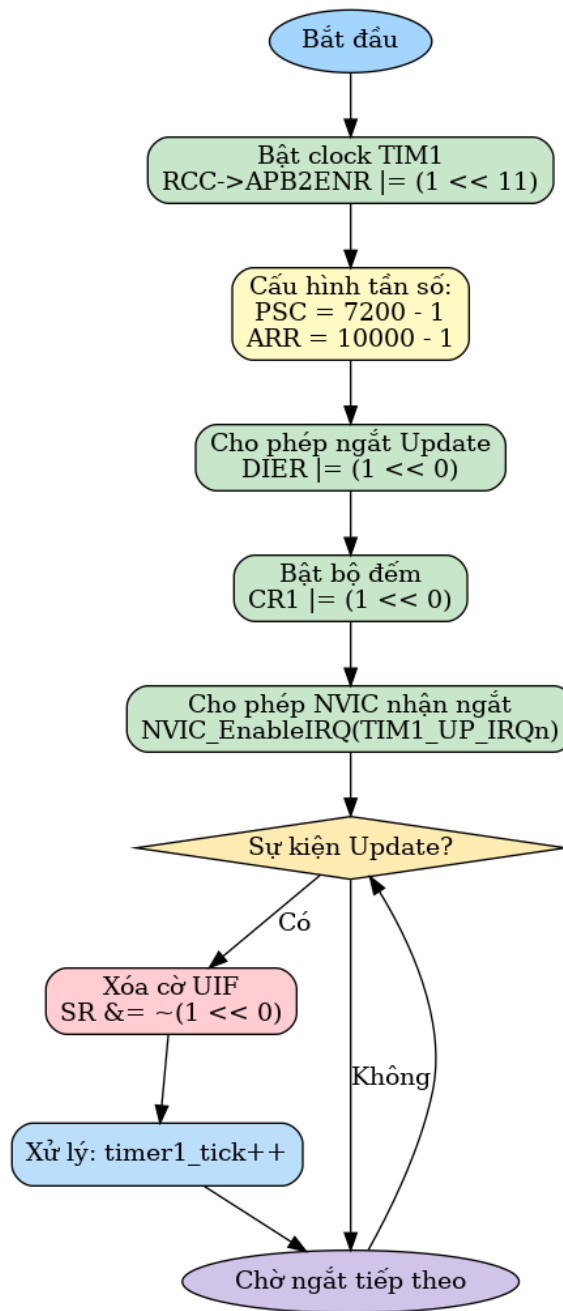
Thanh ghi SR (Status Register)

- **UIF** (Update Interrupt Flag, bit 0):
 - 1: Sự kiện update xảy ra (bộ đếm tràn)
 - Để xóa cờ: ghi 0 vào bit này (ghi 1 sẽ không xóa được).

5.4 Thuật toán

- Bắt đầu
- Bật clock TIM1 (RCC->APB2ENR |= (1 << 11))
- Cấu hình tần số
- $PSC = 7200 - 1$
- $ARR = 10000 - 1$
- Cho phép ngắt Update (DIER |= (1 << 0))
- Bật bộ đếm (CR1 |= (1 << 0))
- Cho phép NVIC nhận ngắt (NVIC_EnableIRQ(TIM1_UP_IRQn))
- Sự kiện Update?
- Không → Quay lại chờ
- Có → Chuyển sang bước 8
- Xóa cờ UIF (SR &= ~(1 << 0))

- Thực hiện xử lý (timer1_tick++)
- Kết thúc vòng lặp / quay lại chờ ngắt



5.5. Ví dụ đầy đủ

```
#include "stm32f10x.h"
```

```
volatile uint8_t tim1_count = 0; // Đếm số lần ngắt
```

```
//-----
```

```

// Khởi tạo TIM1: 1 giây / 1 lần ngắt
//-----

void TIM1_Init(void) {
    // Bật clock cho TIM1
    RCC->APB2ENR |= (1 << 11);    // RCC_APB2ENR_TIM1EN
    // PSC và ARR giữ nguyên như yêu cầu
    TIM1->PSC = 9999;              // Prescaler
    TIM1->ARR = 7200;              // Auto-reload
    // Cho phép ngắt update
    TIM1->DIER |= TIM_DIER_UIE;
    // Bật TIM1
    TIM1->CR1 |= TIM_CR1_CEN;
    // Cho phép ngắt TIM1 trong NVIC
    NVIC_EnableIRQ(TIM1_UP_IRQn);
}
//-----

// Ngắt TIM1
//-----

void TIM1_UP_IRQHandler(void) {
    if (TIM1->SR & TIM_SR_UIF) {    // Kiểm tra cờ update
        TIM1->SR &= ~TIM_SR_UIF;    // Xóa cờ

        tim1_count++;
        if (tim1_count >= 3) {      // Đủ 3 giây
            tim1_count = 0;
            GPIOC->ODR ^= (1 << 13); // Đảo trạng thái PC13
        }
    }
}

//-----

```

```

// Chương trình chính
//-----
int main(void) {
    // Bật clock cho GPIOC
    RCC->APB2ENR |= (1 << 4);    // IOPCEN
    // PC13 output push-pull, 2 MHz
    GPIOC->CRH &= ~(0xF << 20);
    GPIOC->CRH |= (0x2 << 20);
    // Khởi tạo Timer1
    TIM1_Init();
    while (1) {
        // Chương trình chính không làm gì
    }
}

```

5.4 Ứng dụng

- Đếm thời gian chính xác.
- Thực hiện tác vụ định kỳ (ví dụ đọc cảm biến, điều khiển).

6. Ngắt I/O (EXTI - External Interrupt)

6.1 Giới thiệu

- EXTI cho phép ngắt khi có thay đổi điện áp trên chân GPIO.
- Thường dùng để bắt sự kiện nút nhấn, cảm biến,...
- Có nhiều line EXTI, ví dụ EXTI line 0 tới 15 tương ứng với chân GPIO 0 tới 15.

6.2.1 Cấu hình EXTI

- Bật clock port GPIO và AFIO.
- Cấu hình chân GPIO làm input.
- Thiết lập port cho EXTI (AFIO->EXTICR).
 - STM32F1 có 4 thanh ghi **AFIO->EXTICR[0..3]**, mỗi thanh ghi quản lý cấu hình ngắt ngoại vi (EXTI) cho 4 chân GPIO theo nhóm EXTI line:
 - EXTICR[0] cho EXTI line 0..3
 - EXTICR[1] cho EXTI line 4..7

- EXTICR[2] cho EXTI line 8..11
- EXTICR[3] cho EXTI line 12..15
- Mỗi EXTI line dùng **4 bit** trong thanh ghi để chọn port tương ứng (Port A, B, C, D, ...).
- EXTI line n tương ứng với chân GPIO số n (ví dụ EXTI line 0 là chân GPIO số 0, EXTI line 1 là chân GPIO số 1, ...).
- Do đó, EXTI line 1 nằm trong EXTICR[0] và chiếm 4 bit bắt đầu từ bit số 4 (vì EXTI line 0 dùng bit 0-3, EXTI line 1 dùng bit 4-7, ...).
- Các giá trị port tương ứng trong 4 bit đó:
 - 0b0000 = Port A
 - 0b0001 = Port B
 - 0b0010 = Port C
 - 0b0011 = Port D
 - ...
- Vì bạn muốn sử dụng chân PB1 (port B, chân số 1), nên phải cấu hình EXTI line 1 để nhận tín hiệu từ chân này. Nếu muốn dùng chân khác, ví dụ PB2 thì phải dùng EXTI line 2 tương ứng.
- // Cấu hình EXTI line 1 kết nối với chân PB1


```
AFIO->EXTICR[0] &= ~(0xF << 4); // Xóa config EXTI1
AFIO->EXTICR[0] |= (0x1 << 4);    // Chọn port B cho EXTI1 (0b0001)
```
- **Bật ngắt EXTI**
 - **EXTI->IMR** (Interrupt Mask Register) là thanh ghi dùng để **cho phép hoặc chặn** các ngắt ngoại vi EXTI tương ứng từng line.
 - Mỗi bit trong **IMR** tương ứng một **EXTI line**.
 - Bit 0 → EXTI line 0
 - Bit 1 → EXTI line 1
 - Bit 2 → EXTI line 2
 - ...
 - Khi bit tương ứng được **set = 1** thì ngắt của line đó được **cho phép (bật)**.
 - Khi bit = 0 thì ngắt của line đó bị **khóa (không hoạt động)**.
 - **EXTI->IMR |= (1 << 1);** // Bật ngắt EXTI line 1

- Chọn cạnh ngắt (rising/falling/both).
 - **EXTI->FTSR** (Falling Trigger Selection Register) dùng để chọn **ngắt trên cạnh xuống** (từ mức cao xuống mức thấp) cho từng EXTI line.
 - **EXTI->RTSR** (Rising Trigger Selection Register) dùng để chọn **ngắt trên cạnh lên** (từ mức thấp lên mức cao) cho từng EXTI line.
 - Mỗi bit trong hai thanh ghi này tương ứng với một EXTI line:
 - Bit 0 → EXTI line 0
 - Bit 1 → EXTI line 1
 - Bit 2 → EXTI line 2
 - ...
- ```
EXTI->FTSR |= (1 << 1); // Ngắt cạnh xuống (falling edge)
EXTI->RTSR &= ~(1 << 1); // Tắt ngắt cạnh lên
```
- Bật NVIC cho EXTI.
    - **NVIC** (Nested Vectored Interrupt Controller) là bộ điều khiển ngắt của vi xử lý Cortex-M3 trên STM32.
    - Hàm `NVIC_EnableIRQ()` được dùng để **bật cho phép ngắt** trên một nguồn ngắt cụ thể trong NVIC.
    - **Tham số EXTI1\_IRQn** là tên định nghĩa của ngắt EXTI line 1 trong file header của STM32. Nó tương ứng với vị trí ngắt trong bảng ngắt NVIC.
    - **Bật ngắt EXTI line 1 trong NVIC**

```
NVIC_EnableIRQ(EXTI1_IRQn);
```
  - Cho phép ngắt toàn cục:
    - `__enable_irq();`
    - Là hàm gọi trong CMSIS (Cortex Microcontroller Software Interface Standard) để **bật (enable) ngắt toàn cục của CPU**.
    - Khi gọi hàm này, CPU cho phép nhận và xử lý các ngắt đã được cấu hình và bật trong NVIC.
    - Nếu không bật ngắt toàn cục, dù bạn cấu hình và bật từng ngắt riêng lẻ, CPU cũng **không phản hồi ngắt nào**.

### 6.2.2 Chương trình xử lý ngắt khi ngắt xảy ra:

**void EXTI1\_IRQHandler(void)** là tên hàm xử lý ngắt (Interrupt Service Routine - ISR) cho nguồn ngắt EXTI line 1 trên STM32.

---

### Giải thích chi tiết:

- Mỗi nguồn ngắt trong STM32 có một hàm ISR tương ứng, tên hàm này được khai báo trong vector table (bảng vector ngắt).
- Khi EXTI line 1 (tương ứng chân GPIO số 1, ví dụ PB1) phát sinh ngắt, CPU sẽ tự động nhảy đến hàm EXTI1\_IRQHandler() để xử lý.
- Bạn cần viết code trong hàm này để:
  - Kiểm tra nguồn ngắt (ví dụ kiểm tra cờ ngắt EXTI line 1).
  - Xử lý sự kiện (ví dụ tăng biến đếm số lần nhấn nút).
  - Xóa cờ ngắt để tránh xử lý lặp lại.
  - EXTI->PR là thanh ghi **Pending Register** — dùng để kiểm tra xem ngắt của EXTI line nào đang chờ xử lý (pending).
  - EXTI\_PR\_PR1 là bit cờ ngắt của **EXTI line 1** (tương ứng với chân PB1).
  - Câu if (EXTI->PR & EXTI\_PR\_PR1) kiểm tra xem cờ ngắt EXTI line 1 có đang được set (nghĩa là có ngắt từ chân PB1 đang chờ xử lý) hay không.
  - Nếu có, bên trong khối if ta cần **xóa cờ ngắt** để báo với bộ phần cứng rằng ta đã xử lý xong ngắt này, tránh bị xử lý lặp lại liên tục.
  - Để xóa cờ ngắt, theo tài liệu STM32, ta cần **viết giá trị 1 vào bit tương ứng trong thanh ghi PR**.

Nên EXTI->PR |= EXTI\_PR\_PR1; là cách viết 1 vào bit đó để xóa cờ ngắt.

### 6.3 Ví dụ: Ngắt nút nhấn PC13 (chân thường dùng nút trên board STM32F103)

```
void EXTI_PC13_Init(void) {
 RCC->APB2ENR |= RCC_APB2ENR_IOPCEN; // Clock port C
 RCC->APB2ENR |= RCC_APB2ENR_AFIOEN; // Clock AFIO

 // Cấu hình PC13 input pull-up
 GPIOC->CRH &= ~(0xF << ((13 - 8) * 4)); // Xóa config cũ
 GPIOC->CRH |= (0x8 << ((13 - 8) * 4)); // Input pull-up/down (0b1000)
 GPIOC->ODR |= (1 << 13); // Kéo lên (pull-up)
```

```

AFIO->EXTICR[3] &= ~(0xF << 4); // Chọn EXTI13 port C

EXTI->IMR |= (1 << 13); // Bật ngắt EXTI13
EXTI->FTSR |= (1 << 13); // Ngắt cạnh xuống (falling edge)

NVIC_EnableIRQ(EXTI15_10_IRQn); // Bật ngắt NVIC cho EXTI 10-15
}

volatile uint32_t press_count = 0;
void EXTI15_10_IRQHandler(void) {
 if (EXTI->PR & (1 << 13)) { // Kiểm tra cờ ngắt PC13
 EXTI->PR |= (1 << 13); // Xóa cờ ngắt
 press_count++;
 }
}

int main(void) {
 EXTI_PC13_Init();
 while(1) {
 // Main loop có thể dùng biến press_count
 }
}

```

---

## 7. Tổng kết

| Loại ngắt      | Vector ISR         | Cách cấu hình chính                    | Ví dụ ứng dụng                     |
|----------------|--------------------|----------------------------------------|------------------------------------|
| <b>SysTick</b> | SysTick_Handler    | Nạp LOAD, bật CTRL, enable IRQ         | Tạo timer tick, delay millisec     |
| <b>Timer 1</b> | TIM1_UP_IRQHandler | Bật clock, PSC, ARR, DIER, NVIC enable | Tạo ngắt định kỳ 1s, đếm thời gian |

| Loại ngắt      | Vector ISR           | Cách cấu hình chính                           | Ví dụ ứng dụng                      |
|----------------|----------------------|-----------------------------------------------|-------------------------------------|
| EXTI<br>(GPIO) | EXTI15_10_IRQHandler | Cấu hình AFIO, EXTI IMR,<br>FTSR, NVIC enable | Ngắt nút nhấn, cảm<br>biến ngoại vi |